

Page 1 - Python Overview & Environment Setup

Python is an interpreted, high-level, general-purpose programming language. Created by Guido van Rossum and first released in 1991, Python's design philosophy emphasizes code readability with the use of significant indentation.

Study Notes and Key Takeaways for Chapter 1: Master the fundamentals before progressing to advanced object-oriented design and design patterns.

Page 2 - Variables and Data Types in Python

Python features dynamic typing. Key built-in data types include integers, floating-point numbers, strings, booleans, and complex numbers. Variables in Python do not require explicit declaration before use.

Study Notes and Key Takeaways for Chapter 2: Master the fundamentals before progressing to advanced object-oriented design and design patterns.

Page 3 - Python Operators and Expressions

Python supports arithmetic operators (+, -, *, /, //, %, **), comparison operators (==, !=, <, >, <=, >=), logical operators (and, or, not), bitwise operators, and assignment operators.

Study Notes and Key Takeaways for Chapter 3: Master the fundamentals before progressing to advanced object-oriented design and design patterns.

Page 4 - Control Flow: Conditionals in Python

Decision making in Python is accomplished using if, elif, and else statements. Python relies on proper indentation to define scope blocks rather than curly braces.

Study Notes and Key Takeaways for Chapter 4: Master the fundamentals before progressing to advanced object-oriented design and design patterns.

Page 5 - Control Flow: Loops and Iterations

Python provides while loops and for loops. The for loop iterates over the items of any sequence such as lists, tuples, or strings. Loop control statements include break, continue, and pass.

Study Notes and Key Takeaways for Chapter 5: Master the fundamentals before progressing to advanced object-oriented design and design patterns.

Page 6 - Functions and Scope in Python

Functions are defined using the `def` keyword. Python supports positional arguments, keyword arguments, default parameter values, and arbitrary argument lists (`*args` and `**kwargs`).

Study Notes and Key Takeaways for Chapter 6: Master the fundamentals before progressing to advanced object-oriented design and design patterns.

Page 7 - Python Data Structures: Lists and Tuples

Lists are mutable ordered sequences enclosed in square brackets. Tuples are immutable ordered sequences enclosed in parentheses. Both support indexing, slicing, and unpacking operations.

Study Notes and Key Takeaways for Chapter 7: Master the fundamentals before progressing to advanced object-oriented design and design patterns.

Page 8 - Python Data Structures: Dictionaries and Sets

Dictionaries store key-value mappings with fast hash-based lookups. Sets are unordered collections of unique elements that support mathematical set operations like union and intersection.

Study Notes and Key Takeaways for Chapter 8: Master the fundamentals before progressing to advanced object-oriented design and design patterns.

Page 9 - String Manipulation and Regular Expressions

Strings in Python are immutable sequences of Unicode characters. String methods include `split`, `join`, `strip`, `replace`, `format`, and f-strings for clean variable interpolation.

Study Notes and Key Takeaways for Chapter 9: Master the fundamentals before progressing to advanced object-oriented design and design patterns.

Page 10 - File Handling and Context Managers

Python provides the `open` function to read and write files. Using the `'with'` statement creates context managers that automatically close resources safely even when exceptions occur.

Study Notes and Key Takeaways for Chapter 10: Master the fundamentals before progressing to advanced object-oriented design and design patterns.

Page 11 - Error and Exception Handling

Exceptions are handled using try, except, else, and finally blocks. Custom exceptions can be created by subclassing the built-in Exception class.

Study Notes and Key Takeaways for Chapter 11: Master the fundamentals before progressing to advanced object-oriented design and design patterns.

Page 12 - Object-Oriented Programming: Inheritance in Python

What is inheritance in Python?

Inheritance is a feature in Python that allows one class to acquire the properties and methods of another class. It helps in code reusability and creating relationships between classes.

The class whose properties and methods are inherited is known as the Parent class (or Superclass / Base class), while the class that inherits those properties is known as the Child class (or Subclass / Derived class).

Syntax Example:

```
class Animal:
    def speak(self):
        return 'Generic animal sound'

class Dog(Animal):
    def speak(self):
        return 'Woof!'
```

Key Benefits of Inheritance:

1. Code Reusability: Common functionality is written once in base classes and shared.
2. Extensibility: Child classes can override methods or add new specialized features.
3. Polymorphic Behavior: Derived classes can be used wherever their base classes are expected.

Page 13 - Polymorphism and Encapsulation

Polymorphism allows methods to perform different actions based on the object calling them.

Encapsulation restricts direct access to some of an object's components using private and protected attributes.